

Лабораторная работа №6

Имитационное моделирование

Чистов Даниил Максимович

Содержание

1	Цель работы	5
2	Выполнение работы	6
2.1	Предварительные работы	6
2.2	Создание необходимых скриптов	7
2.3	Кратко об SIR	7
2.4	Базовый прогон модели	14
2.5	Исследование коэффициента заражения	17
2.6	Создание анимации детерминированной динамики	20
2.7	Итоговый отчёт	21
3	Выводы	25
4	Список литературы	26

Список иллюстраций

2.1	Рабочее пространство для лабораторной работы	6
2.2	Скрипт в папке /src	7
2.3	Скрипты в папке /scripts	7
2.4	Детерминистический график динамики модели SIR	16
2.5	Стохастический график динамики модели SIR	17
2.6	График коэффициента заражения β	19
2.7	График сравнения детерминированной и стохастической симуляций	23
2.8	График $\text{peak}_I(\beta)$	24

Список таблиц

```
import Pkg
Pkg.activate("../project")
Pkg.instantiate()
```

Activating project at `C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab06\project`

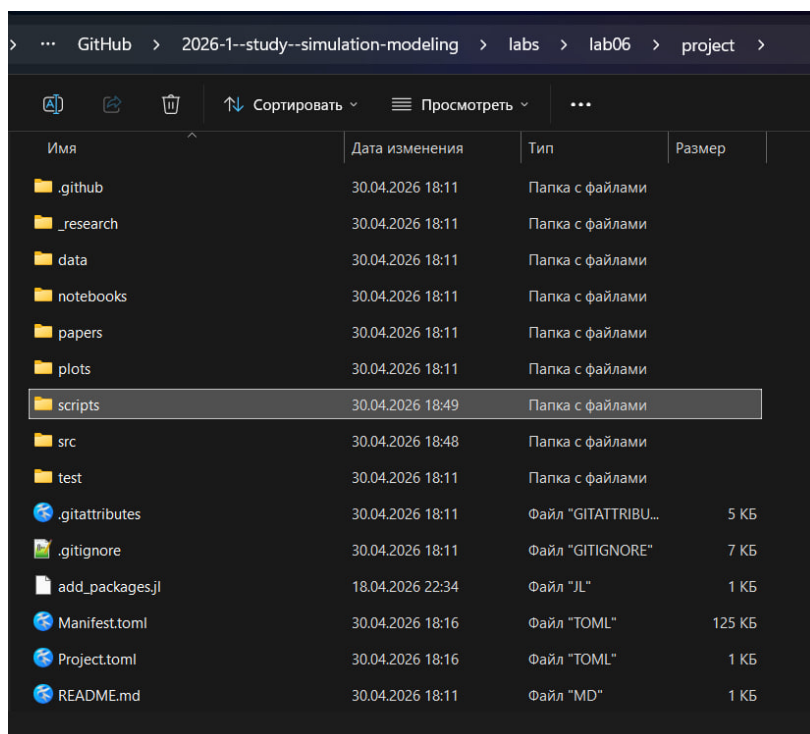
1 Цель работы

Целью данной работы является изучение реализации модели SIR с помощью сетей Петри.

2 Выполнение работы

2.1 Предварительные работы

Перед началом работы с моделью создаю рабочее пространство для лабораторной работы (рис. 2.1).



The screenshot shows a file explorer window with the following path: > ... GitHub > 2026-1--study--simulation-modeling > labs > lab06 > project >. The interface includes a toolbar with icons for file operations and a table listing the contents of the directory.

Имя	Дата изменения	Тип	Размер
.github	30.04.2026 18:11	Папка с файлами	
_research	30.04.2026 18:11	Папка с файлами	
data	30.04.2026 18:11	Папка с файлами	
notebooks	30.04.2026 18:11	Папка с файлами	
papers	30.04.2026 18:11	Папка с файлами	
plots	30.04.2026 18:11	Папка с файлами	
scripts	30.04.2026 18:49	Папка с файлами	
src	30.04.2026 18:48	Папка с файлами	
test	30.04.2026 18:11	Папка с файлами	
.gitattributes	30.04.2026 18:11	Файл "GITATTRIBU..."	5 КБ
.gitignore	30.04.2026 18:11	Файл "GITIGNORE"	7 КБ
add_packages.jl	18.04.2026 22:34	Файл ".jl"	1 КБ
Manifest.toml	30.04.2026 18:16	Файл "TOML"	125 КБ
Project.toml	30.04.2026 18:16	Файл "TOML"	1 КБ
README.md	30.04.2026 18:11	Файл "MD"	1 КБ

Рисунок 2.1: Рабочее пространство для лабораторной работы

2.2 Создание необходимых скриптов

Для выполнения работы требовалось использовать предоставленные скрипты. Каждый скрипт будет описан позже. Создаю в папке /src исходный код модели: SIRPetri (рис. 2.2).

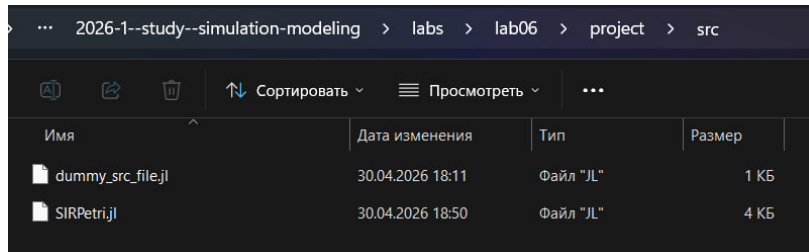


Рисунок 2.2: Скрипт в папке /src

В папке /scripts создаю оставшиеся необходимые скрипты (рис. 2.3).

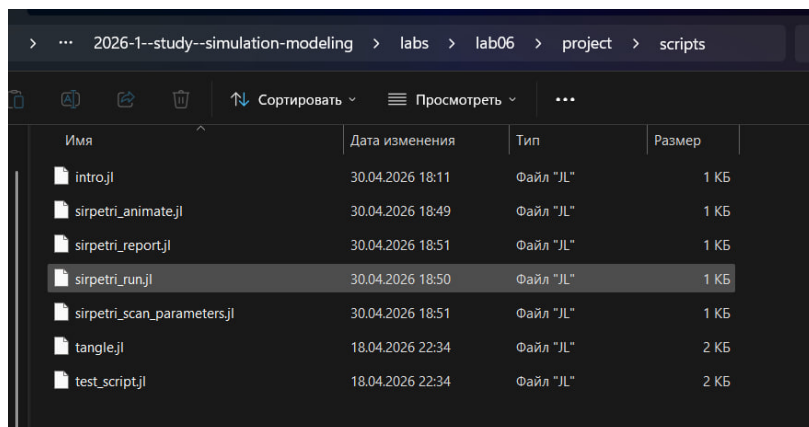


Рисунок 2.3: Скрипты в папке /scripts

2.3 Кратко об SIR

Данная модель уже была разобрана в прошлых лабораторных работах. Если кратко её описывать, то модель эпидемии SIR (Susceptible–Infectious–Recovered) с использованием сетей Петри описывает переходы между тремя состояниями:

- S (восприимчивые) - могут заразиться;

- I (инфицированные) - заражают других и выздоравливают;
- R (выздоровевшие / с иммунитетом) — больше не участвуют в эпидемии.

Сеть Петри содержит два перехода:

- infection: $S + I \rightarrow I + I$ (скорость β);
- recovery: $I \rightarrow R$ (скорость γ).

Основной код модели, который используется остальными программами: src/SIR-Petri.jl

Содержит функции построения сетей Петри, детерминированной симуляции, стохастической симуляции, визуализации

```
module SIRPetri

using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random

export build_sir_network, simulate_deterministic, simulate_stochastic
export plot_sir, to_graphviz_sir

"""
    build_sir_network(β=0.3, γ=0.1)

    build_sir_network(β=0.3, γ=0.1)
    build_sir_network(β=0.3, γ=0.1) SIR.
```



```
S, I, R = u
β, γ = rates
```

Пропускные способности (закон действующих масс)

```
infection_rate = β * S * I
recovery_rate = γ * I
```

Изменения

```
du[1] = -infection_rate          # dS/dt
du[2] = infection_rate - recovery_rate # dI/dt
du[3] = recovery_rate            # dR/dt
end
return f!
end

"""
    simulate_deterministic(net, u0, tspan; saveat=0.1, rates=[0.3,0

#####  ##### ODE-#####.
##### DataFrame  ##### time, S, I, R.
"""

function simulate_deterministic(net, u0, tspan; saveat = 0.1, rates
    f = sir_ode(net, rates)
    prob = ODEProblem(f, u0, tspan)
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    df.S = sol[1, :]
    df.I = sol[2, :]
```

```

    df.R = sol[3, :]
    return df
end

"""
    simulate_stochastic(net, u0, tspan; rates=[0.3,0.1], rng=Random

    Возвращает DataFrame с колонками S, I, R.
    DataFrame.
"""
function simulate_stochastic(net, u0, tspan; rates = [0.3, 0.1], rng=
    u = copy(u0)
    t = 0.0
    times = [t]
    states = [copy(u)]
    λ, μ = rates
    while t < tspan[2]
        S, I, R = u
        a_inf = λ * S * I
        a_rec = μ * I
        a0 = a_inf + a_rec
        if a0 == 0
            break
        end
        dt = -log(rand(rng)) / a0
        r = rand(rng) * a0
        if r < a_inf

```

Заражение

```

        u[1] -= 1
        u[2] += 1
    else

```

Выздоровление

```

        u[2] -= 1
        u[3] += 1
    end
    t += dt
    if t <= tspan[2]
        push!(times, t)
        push!(states, copy(u))
    end
end

df = DataFrame(time = times)
df.S = [s[1] for s in states]
df.I = [s[2] for s in states]
df.R = [s[3] for s in states]
return df
end

"""
    plot_sir(df)

    XXXXXX XXXXXX XXXXXXXXXX S, I, R DataFrame.
    """

function plot_sir(df)
    p = plot(

```

```

        df.time,
        [df.S, df.I, df.R],
        label = ["S (Susceptible)" "I (Infected)" "R (Recovered)"],
        xlabel = "Time",
        ylabel = "Population",
        linewidth = 2,
    )
    return p
end

"""
    to_graphviz_sir(net)

    Convert the Petri net to a Graphviz graph.

    """
function to_graphviz_sir(net)
    return to_graphviz(net, prog = "dot")
end

end # module

```

Функция: `build_sir_network( $\beta$ ,  $\gamma$ )`

Создаёт размеченную сеть Петри (LabelledPetriNet) с двумя переходами:

- infection: $S + I \rightarrow I + I$ (скорость β)
- recovery: $I \rightarrow R$ (скорость γ)

Возвращает:

- объект сети `net`,

- начальную маркировку $u_0 = [990.0, 10.0, 0.0]$,
- имена состояний $[:S, :I, :R]$.

Функция `simulate_deterministic(net, u0, tspan; saveat, rates)`:

- Строит систему обыкновенных дифференциальных уравнений (ОДУ) на основе закона действующих масс.
- Решает её численно с помощью `OrdinaryDiffEq` (метод `Tsit5`).
- Возвращает `DataFrame` с колонками `time`, `S`, `I`, `R`.

Функция `simulate_stochastic(net, u0, tspan; rates, rng)`

- Используется алгоритм Гиллеспи.
- Реализует прямой метод Гиллеспи (SSA) для дискретных событий.

На каждом шаге вычисляет пропускные способности:

- $a_{\text{inf}} = \beta * S * I$
- $a_{\text{rec}} = \gamma * I$
- Случайным образом выбирает, произойдёт ли заражение или выздоровление, и через какое время.
- Возвращает `DataFrame` с временами и целочисленными маркировками.

2.4 Базовый прогон модели

Данный код выполняет один базовый эксперимент с фиксированными параметрами $\beta = 0.3$, $\gamma = 0.1$, запускает детерминированную симуляцию (решение ОДУ) — даёт плавную усреднённую динамику, и стохастическую симуляцию (алгоритм Гиллеспи) — учитывает случайные флуктуации. В конце код сохраняет результаты в CSV-файлы и строит графики $S(t)$, $I(t)$, $R(t)$ для обоих типов симуляции.

Входные данные модели:

- $\beta = 0.3$ (коэффициент заражения)
- $\gamma = 0.1$ (коэффициент выздоровления)
- $t_{\max} = 100.0$ (время симуляции)
- Начальная маркировка сети Петри: $S_0 = 990$, $I_0 = 10$, $R_0 = 0$.
- Зерно для генератора случайных чисел: `Random.seed!(123)` (обеспечивает воспроизводимость стохастического прогона).

Ниже предоставлен код файла `sirpetri_run.jl`

```
using DrWatson
@quickactivate "project"
using Random
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

Параметры

```
β = 0.3
γ = 0.1
tmax = 100.0
```

Создаём сеть

```
net, u0, states = build_sir_network(β, γ)
```

Детерминированная симуляция

```
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
CSV.write(datadir("sir_det.csv"), df_det)
```

Стохастическая симуляция

```
Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [λ, μ])
CSV.write(datadir("sir_stoch.csv"), df_stoch)
```

Графики

```
p_det = plot_sir(df_det)
savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))

println("XXXXXXXX XXXXXX XXXXXXXXXXX. XXXXXXXXXXXX X data/ X plots/")
```

В результате были получены следующие графики:

- Детерминистический график (рис. 2.4).
- Стохастический график (рис. 2.5).

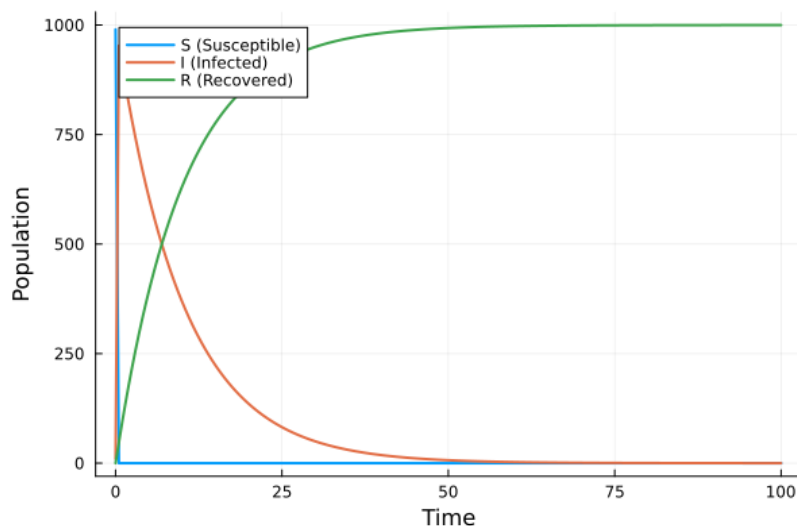


Рисунок 2.4: Детерминистический график динамики модели SIR

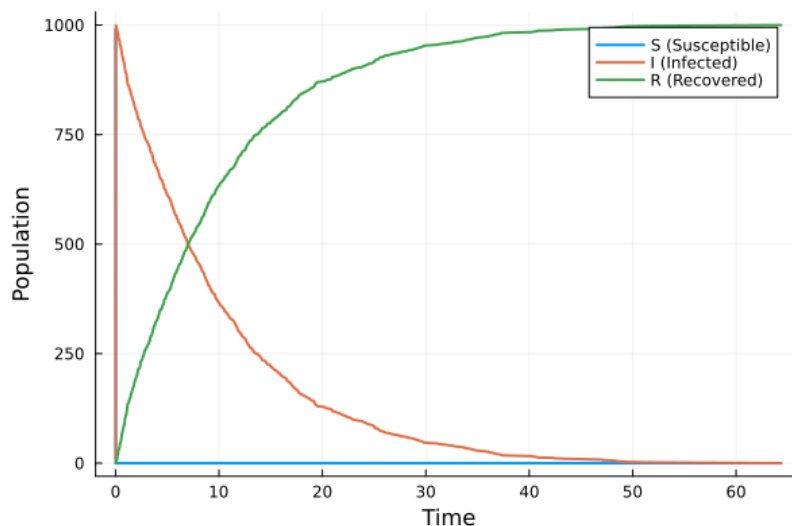


Рисунок 2.5: Стохастический график динамики модели SIR

Проанализировав графики можно сделать следующие выводы:

- в детерминированном графике прослеживается классический пик эпидемии: рост I, максимум, затем спад до нуля; R растёт и выходит на плато, S падает.
- Стохастический график может иметь шумы и, возможно, немного отличаться по времени пика и амплитуде – это демонстрирует влияние случайности.

Сравнив обе симуляции, можно сказать, что при большом начальном числе восприимчивых (990) стохастическая траектория близка к детерминированной, но при малых числах различия были бы значительны.

2.5 Исследование коэффициента заражения

Данный код исследует чувствительность модели к изменению параметра β (скорости заражения) следующим образом: для каждого значения β из диапазона 0.1:0.05:0.8 запускается детерминированная симуляция ($\gamma = 0.1$ фиксирован). Для каждого прогона вычисляются максимальное число инфицированных (пик эпидемии) – `peak_I` и конечное число выздоровевших – `final_R`. Затем результаты сводятся в таблицу и строится график зависимости `peak_I( $\beta$ )` и `final_R( $\beta$ )`

Входные данные эксперимента:

- $\beta_range = 0.1 : 0.05 : 0.8$ (всего 15 значений)
- $\gamma_fixed = 0.1$
- $tmax = 100.0$

Остальные параметры такие же, как в базовом прогоне.

Ниже предоставлен код файла `sirpetri_scan_parameters.jl`

```
using DrWatson
@quickactivate "project"
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

Диапазон β

```
 $\beta\_range = 0.1:0.05:0.8$ 
 $\beta\_fixed = 0.1$ 
 $tmax = 100.0$ 

results = []
for  $\beta$  in  $\beta\_range$ 
    net, u0, _ = build_sir_network( $\beta$ ,  $\beta\_fixed$ )
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, ( $\beta$  =  $\beta$ , peak_I = peak_I, final_R = final_R))
end
```

```
df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)
```

График

```
p = plot(
    df_scan[:,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I" "Final R"],
    marker = :circle,
    xlabel = " $\beta$  (infection rate)",
    ylabel = "Population",
)
savefig(plotsdir("sir_scan.png"))

println("XXXXXXXXXXXX  $\beta$  XXXXXXXX. XXXXXXXX  $\beta$  data/sir_scan.csv")
```

В результате был получен следующий график: (рис. 2.6).

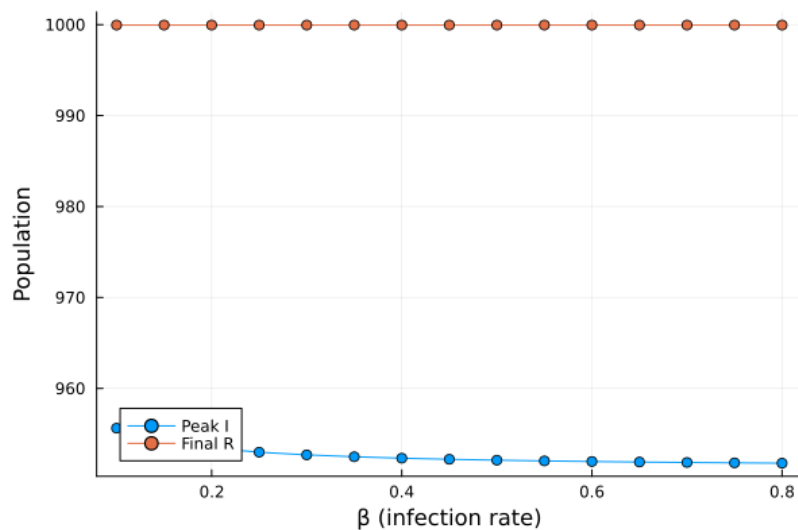


Рисунок 2.6: График коэффициента заражения β

2.6 Создание анимации детерминированной динамики

Данный код проводит идентичный базовый прогон, что и в `sirpetri_run`, однако он фиксирует каждый момент и создаёт анимацию в виде GIF файла. Во время выполнения работы возникли трудности с интеграцией кода, поэтому потребовалось использовать код из предыдущей лабораторной работы для создания анимации.

Ниже предоставлен код файла `sirpetri_animate.jl`

```
using DrWatson
@quickactivate "project"
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

Параметры

```
β = 0.3
γ = 0.1
tmax = 100.0
```

Создаём сеть

```
net, u0, states = build_sir_network(β, γ)

df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
state_cols = [col for col in propertynames(df_det) if col != :time])

ymax = maximum(Matrix(df_det[:, state_cols]))
```

```

anim = @animate for i in 1:nrow(df_det)
    t = df_det.time[1:i]
    y = Matrix(df_det[1:i, state_cols])
    plot(
        t,
        y,
        label = reshape(["S (Susceptible)", "I (Infected)", "R (Recovered)"], nrow(y)),
        xlabel = "Time",
        ylabel = "Population",
        title = "SIR dynamics at t = $(round(df_det.time[i], digits=1))",
        xlims = (0, tmax),
        ylims = (0, ymax + 10),
        linewidth = 2,
        legend = :topright
    )
end

gif(anim, plotsdir("sir_animation.gif"), fps = 10)
println("XXXXXXXX XXXXXXXXXX X plots/sir_animation.gif")

```

Код успешно работает, с результатом вы можете ознакомиться в папке `lab06/project/plots/sir_animation.gif`

2.7 Итоговый отчёт

Данный код загружает ранее сохранённые результаты (`sir_det.csv`, `sir_stoch.csv`, `sir_scan.csv`), которые были получены при запуске предыдущих скриптов, и строит сравнительные графики для итогового отчёта:

- Сравнение детерминированной и стохастической динамики инфицированных $I(t)$.
- Зависимость пика I от β (результат сканирования).

Ниже предоставлен код файла `sirpetri_report.jl`

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)
```

Сравнение детерминированной и стохастической динамики

```
p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
)
savefig(plotsdir("comparison.png"))
```

Зависимость пика I от β

```
p2 = plot(
    df_scan.β,
    df_scan.peak_I,
```

```

    marker = :circle,
    xlabel = "t",
    ylabel = "Peak I",
    title = "Sensitivity",
)
savefig(plotsdir("sensitivity.png"))

println("XXXXXXXXXX XXXXXXXX XXXXXXXXXXXX X plots/")

```

В результате были получены следующие графики:

- График сравнения детерминированной и стохастической симуляций (рис. 2.7).
- График $\text{peak}_I(\beta)$ (те же данные, что и в `sir_scan.png` (рис. 2.8).

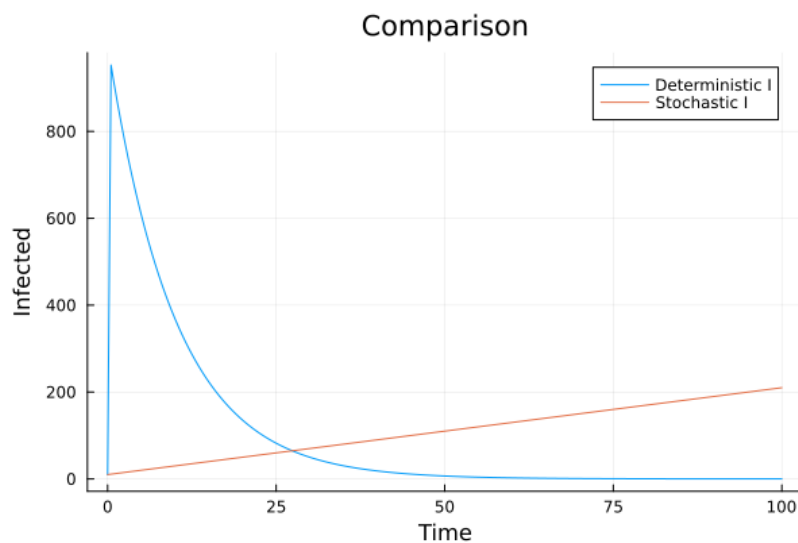


Рисунок 2.7: График сравнения детерминированной и стохастической симуляций

Сравнение детерминированной и стохастической кривых $I(t)$ показывает, насколько велик разброс, вызванный случайностью. При большом размере популяции они близки

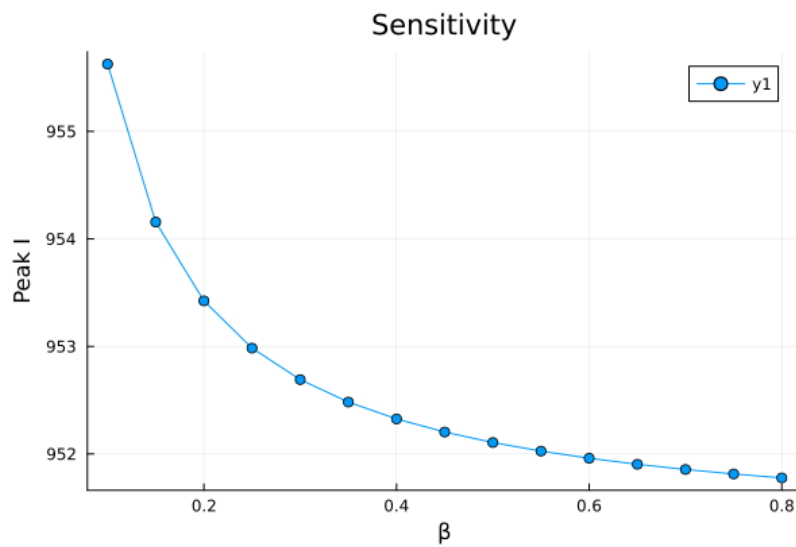


Рисунок 2.8: График $\text{peak}_I(\beta)$

График чувствительности позволяет количественно оценить, как изменение заразности β влияет на тяжесть эпидемии (пик I). Таким образом, это может помочь в принятии решений - например, меры по снижению β .

3 Выводы

В результате выполнения данной лабораторной работы я укрепил свои знания сетей Петри и модели SIR, изучив реализацию модели SIR с помощью сетей Петри

4 Список литературы

Лабораторная работа №6